

Assignment – Module 4: Codex In-depth (Tool Mastery)

Name	SAGAR KUMAR
PRN	SOE25BTAM29
Section	A
Institute	Intellipaat School of Technology, Pune
Subject	Generative AI
Submission	Soft copy (LMS)

Tool used: OpenAI Codex (CLI / VS Code extension) | **Language:** Python 3 | **Testing:** pytest

Q1. Build a Student Grade Calculator with Codex

Prompt given to Codex:

Create a Python console program `grade_calculator.py` called “Student Grade Calculator”. It should:

1. Accept marks for 5 subjects (Maths, Physics, Chemistry, English, Computer).
 2. Calculate total, average and percentage.
 3. Show PASS if every subject is 40 or more, otherwise FAIL.
 4. Reject marks below 0 or above 100, and non-numeric input, and ask again.
- Keep the code simple with small functions and comments.

Code generated by Codex (`grade_calculator.py`):

```
SUBJECTS = ["Maths", "Physics", "Chemistry", "English", "Computer"]
PASS_MARK = 40
```

```
def get_mark(subject):
    while True:
        text = input(f"Enter marks for {subject} (0-100): ")
        try:
            mark = float(text)
        except ValueError:
            print(" Invalid input. Please enter a number.")
            continue
        if mark < 0 or mark > 100:
            print(" Invalid marks. Marks must be between 0 and 100.")
            continue
        return mark
```

```
def calculate(marks):
    total = sum(marks)
    average = total / len(marks)
    percentage = total / (len(marks) * 100) * 100
    passed = all(m >= PASS_MARK for m in marks)
    return total, average, percentage, passed

def main():
    print("=== Student Grade Calculator ===")
    marks = [get_mark(s) for s in SUBJECTS]
    total, average, percentage, passed = calculate(marks)
    print(f"Total Marks : {total:.0f} / 500")
    print(f"Average      : {average:.2f}")
    print(f"Percentage     : {percentage:.2f}%")
    print(f"Result         : {'PASS' if passed else 'FAIL'}")

if __name__ == "__main__":
    main()
```

1. Run it: python grade_calculator.py

2. Test with valid marks (78, 85, 69, 90, 88):

```
=== Student Grade Calculator ===
Enter marks for Maths (0-100): 78
Enter marks for Physics (0-100): 85
Enter marks for Chemistry (0-100): 69
Enter marks for English (0-100): 90
Enter marks for Computer (0-100): 88
Total Marks : 410 / 500
Average      : 82.00
Percentage   : 82.00%
Result       : PASS
```

3. Test with invalid marks (120, -5, "abc", then valid marks with one subject below 40):

```
=== Student Grade Calculator ===
Enter marks for Maths (0-100): 120
    Invalid marks. Marks must be between 0 and 100.
Enter marks for Maths (0-100): -5
    Invalid marks. Marks must be between 0 and 100.
Enter marks for Maths (0-100): abc
    Invalid input. Please enter a number.
Enter marks for Maths (0-100): 35
Enter marks for Physics (0-100): 50
Enter marks for Chemistry (0-100): 60
Enter marks for English (0-100): 70
Enter marks for Computer (0-100): 80
```

Total Marks : 295 / 500
Average : 59.00
Percentage : 59.00%
Result : FAIL

The program did not crash on wrong input. It showed a message and asked again. The result is FAIL because Maths (35) is below 40.

4. Codex's explanation of the important parts (summarised):

- **SUBJECTS** and **PASS_MARK** are constants at the top, so subjects or the pass mark can be changed in one place.
 - **get_mark()** uses a while True loop with try/except ValueError to catch text input, and an if check to reject marks outside 0–100. It only returns when the mark is valid.
 - **calculate()** uses sum() for the total and divides by the number of subjects for the average. The percentage is $\text{total} \div \text{maximum marks} \times 100$. all() checks that **every** subject is at least 40.
 - **main()** collects marks using a list comprehension and prints the result in a neat format with 2 decimal places.
 - **if __name__ == "__main__":** runs main() only when the file is run directly, not when it is imported (useful for testing).
-

Q2. Add a Feature to the Student Task Manager (Due Date)

Step 1 – Inspect the project.

Prompt: “Inspect this Student Task Manager project and explain its structure and current features.”

Codex: The project has task_manager.py with a tasks list and the functions add_task(), complete_task(), delete_task() and list_tasks(). Each task is a dictionary with title and done.

Step 2 – Which files need to change?

Prompt: “I want to add a due-date feature. Which files need to change?”

Codex: Only task_manager.py (task data and display) needs to change, plus a new test file test_task_manager.py.

Step 3 – Implementation plan.

Prompt: “Give me an implementation plan before writing code. Do not remove existing features.”

Codex's plan:

1. Add an **optional** due_date parameter (format YYYY-MM-DD) to add_task(). Default is None, so old calls still work.
2. Validate the date format. Raise ValueError if it is wrong.

3. Add an `is_overdue()` helper: a task is overdue if it has a due date in the past and is not done.
4. Show the due date and an "OVERDUE" flag in `list_tasks()`.
5. Add tests for both the new and the old behaviour.

Step 4 – Implement. Updated `task_manager.py`:

```
from datetime import datetime, date

tasks = []

def add_task(title, due_date=None):
    if due_date:
        datetime.strptime(due_date, "%Y-%m-%d") # raises ValueError
    if wrong format
        tasks.append({"title": title, "done": False, "due_date":
due_date})

def complete_task(index):
    tasks[index]["done"] = True

def delete_task(index):
    tasks.pop(index)

def is_overdue(task, today=None):
    if not task["due_date"] or task["done"]:
        return False
    today = today or date.today()
    return datetime.strptime(task["due_date"], "%Y-%m-%d").date() <
today

def list_tasks(today=None):
    lines = []
    for i, t in enumerate(tasks, start=1):
        status = "Done" if t["done"] else "Pending"
        due = t["due_date"] or "No due date"
        flag = " <-- OVERDUE" if is_overdue(t, today) else ""
        lines.append(f"{i}. {t['title']} [{status}] Due: {due}{flag}")
    return lines
```

Step 5 – Test. `test_task_manager.py`:

```
from datetime import date
import task_manager as tm
```

```
def setup_function():
    tm.tasks.clear()

def test_add_without_due_date_still_works():
    tm.add_task("Read notes")
    assert tm.tasks[0]["due_date"] is None

def test_add_with_due_date():
    tm.add_task("Submit assignment", "2026-10-07")
    assert tm.tasks[0]["due_date"] == "2026-10-07"

def test_invalid_date_rejected():
    try:
        tm.add_task("Bad", "07-10-2026")
        assert False
    except ValueError:
        pass

def test_overdue_flag():
    tm.add_task("Old task", "2026-10-01")
    assert tm.is_overdue(tm.tasks[0], today=date(2026, 10, 6))

def test_completed_task_not_overdue():
    tm.add_task("Old task", "2026-10-01")
    tm.complete_task(0)
    assert not tm.is_overdue(tm.tasks[0], today=date(2026, 10, 6))

def test_delete_still_works():
    tm.add_task("A")
    tm.delete_task(0)
    assert tm.tasks == []
```

Test result:

```
$ python -m pytest test_task_manager.py -q
..... [100%]
6 passed
```

Step 6 – Review the changes.

- **Existing features kept:** add, complete, delete and list still work. due_date is optional, so old code like add_task("Read notes") still works (tested).
- **New feature:** due date with format validation and an overdue warning.

- **Good design choice:** today can be passed into `is_overdue()`, which makes the tests stable (they do not depend on the real date).
 - **Possible future improvement:** sort tasks by due date and save tasks to a file.
-

Q3. Debug a Broken Application

(Example application used for practice. The same workflow applies to the program given by the instructor.)

Workflow: Reproduce – Understand the Error – Find Root Cause – Implement Fix – Test – Check Side Effects

Original problem (average_buggy.py): A program that calculates the average of student marks crashes for some inputs.

```
def average(marks):  
    total = 0  
    for m in marks:  
        total += m  
    return total / len(marks)
```

```
print(average([80, 90, 70]))  
print(average([]))
```

1. Reproduce: python average_buggy.py

2. Error observed:

```
80.0  
Traceback (most recent call last):  
  File "average_buggy.py", line 8, in <module>  
    print(average([]))  
  File "average_buggy.py", line 5, in average  
    return total / len(marks)  
ZeroDivisionError: division by zero
```

3. Root cause identified by Codex:

When the list is **empty**, `len(marks)` is **0**, and Python cannot divide by zero. The function never checks for an empty list. (This happens, for example, when a student has no marks entered yet.)

4. Fix:

```
def average(marks):  
    if not marks:                # handle empty list  
        return 0  
    return sum(marks) / len(marks)
```

(Codex also simplified the loop to `sum()`.)

5. Test result:

80.0
0

Normal input still gives 80.0, and an empty list now returns 0 instead of crashing.

6. Side effects checked:

- Normal lists give the **same result as before** (80.0). ✓
 - Single-value list [50] → 50.0. ✓
 - **One point to note:** returning 0 for “no marks” could be confused with a real average of 0. Codex suggested that, if needed, the function could return None and the caller could show “No marks available”. No other functions were affected.
-

Q4. Refactor and Test Existing Code

(Example program used for practice. The same steps apply to the program given by the instructor.)

Original (poorly written) code – report_old.py:

```
def report(m1, m2, m3):  
    if m1 >= 90:  
        g1 = "A"  
    elif m1 >= 75:  
        g1 = "B"  
    elif m1 >= 40:  
        g1 = "C"  
    else:  
        g1 = "F"  
    if m2 >= 90:  
        g2 = "A"  
    elif m2 >= 75:  
        g2 = "B"  
    elif m2 >= 40:  
        g2 = "C"  
    else:  
        g2 = "F"  
    if m3 >= 90:  
        g3 = "A"  
    elif m3 >= 75:  
        g3 = "B"  
    elif m3 >= 40:  
        g3 = "C"  
    else:  
        g3 = "F"  
    return [g1, g2, g3]
```

1. Understanding (Codex): The function takes 3 marks and converts each into a letter grade: A (≥ 90), B (≥ 75), C (≥ 40), F (< 40). It returns a list of 3 grades.

2. Problems found:

- The **same grading logic is copied 3 times** (duplicated code).
- Unclear variable names (m1, g1...).
- It only works for exactly 3 subjects.
- If the grading rules change, they must be changed in 3 places, which can easily cause mistakes.

3. Refactoring plan:

1. Move the grading logic into one function, `get_grade(mark)`.
2. Make `report()` accept any number of marks.
3. Add docstrings.
4. Write tests to prove the behaviour is the same.

4. Refactored code – `report_new.py`:

```
def get_grade(mark):  
    """Return the letter grade for one mark."""  
    if mark >= 90:  
        return "A"  
    if mark >= 75:  
        return "B"  
    if mark >= 40:  
        return "C"  
    return "F"  
  
def report(*marks):  
    """Return a list of grades, one for each mark."""  
    return [get_grade(m) for m in marks]
```

5. Tests for behaviour and edge cases – `test_report.py`:

```
import report_old, report_new  
  
def test_grade_boundaries():  
    assert report_new.get_grade(90) == "A"  
    assert report_new.get_grade(89) == "B"  
    assert report_new.get_grade(75) == "B"  
    assert report_new.get_grade(74) == "C"  
    assert report_new.get_grade(40) == "C"  
    assert report_new.get_grade(39) == "F"  
    assert report_new.get_grade(0) == "F"
```

```
def test_same_behaviour_as_old_code():
    for a in range(0, 101, 7):
        for b in (0, 39, 40, 75, 90, 100):
            assert report_new.report(a, b, 60) == report_old.report(a,
b, 60)
```

6. Run the tests:

```
$ python -m pytest test_report.py -q
..                                     [100%]
2 passed
```

7. Behaviour unchanged? Yes. The second test compares the old and new functions for many mark combinations, and they give exactly the same output. The boundary tests (39/40, 74/75, 89/90) confirm the grade limits did not change. The code is now shorter (from 25 lines to 12), easier to read, and has the grading rule in only one place.

Q5. Create and Use AGENTS.md

AGENTS.md for the Student Task Manager:

AGENTS.md – Student Task Manager

Project Information

- A simple command-line Student Task Manager written in Python 3.
- Main file: task_manager.py | Tests: test_task_manager.py
- Features: add task, complete task, delete task, list tasks, due dates.

Coding Rules

- Follow PEP 8 style; use 4 spaces for indentation.
- Use clear snake_case names for functions and variables.
- Keep functions small (one job per function) and add a short docstring.
- Do not use external libraries; only the Python standard library.
- Dates must use the format YYYY-MM-DD.

Testing Instructions

- Run all tests with: python -m pytest -q
- Every new feature must have at least one new test.
- All tests must pass before the task is complete.

Project Constraints

- Do NOT remove or rename existing functions or features.
- Do not change the format of existing task dictionaries; only add new keys.

- Keep it a command-line app (no web frameworks, no database).
- After any change, explain which files changed and why.

Small task given to Codex:

“Following AGENTS.md, add a feature to **count pending tasks**.”

What Codex did:

```
def count_pending():  
    """Return the number of tasks that are not done."""  
    return sum(1 for t in tasks if not t["done"])
```

New test added:

```
def test_count_pending():  
    tm.add_task("A")  
    tm.add_task("B")  
    tm.complete_task(0)  
    assert tm.count_pending() == 1
```

Test run: `python -m pytest -q` → **7 passed**.

Codex’s explanation: “I added one function, `count_pending()`, to `task_manager.py` and one test to `test_task_manager.py`. No existing functions were changed or removed. Only the standard library was used. All tests pass.”

Final check:

Question	Answer
Did Codex follow the instructions?	Yes. It used snake_case, added a docstring, used only the standard library and kept the function small.
Did it preserve existing features?	Yes. add, complete, delete, list and due-date code were untouched. The old tests still pass.
Did it run the tests?	Yes. It ran <code>python -m pytest -q</code> : 7 passed.
Did it explain the changes?	Yes. It listed the changed files and the reason, as required by AGENTS.md.

Learning: AGENTS.md works like a **rule book for the AI**. I do not need to repeat the same instructions in every prompt, and Codex’s output stays consistent with the project’s style and rules.